Project: Development of an Agentic RAG Chatbot Prototype

The task is to develop an **Agentic RAG (Retrieval-Augmented Generation)**–based chatbot prototype in **Python**, using the **LangGraph framework**. The goal is to deliver a **functional, well-documented, and reproducible solution** that demonstrates confident knowledge of **agentic system design** and the **integration of modular RAG subsystems**.

Problem Definition and Data Source

Problem Selection

Select a real-world problem (domain/use case) for which the chatbot provides a solution.

Justification

In the documentation, briefly justify your choice based on the following aspects:

- Why is the problem relevant?
- What user needs does it address?
- Why is the agentic RAG approach advantageous for this problem?

Architecture and Agentic Behavior

Agentic Workflow (LangGraph)

Create an agentic workflow using the **LangGraph framework** that contains **at least 5 nodes**. Your graph must include:

- Autonomous decision-making (e.g. conditional routing)
- Decomposition into subtasks and independent execution
- State management for storing intermediate results

Tools

Integrate **at least 2 tools** into the workflow. In addition to RAG functionality, there must be **at least one tool that is not purely retrieval-based**.

RAG Subsystem

Create a **dedicated, modular RAG subgraph** that can be invoked from the main workflow but **does not count toward the 3–5 nodes** requirement.

Data Source

Use a freely chosen **text-based data source** (e.g. PDF documents, public datasets, articles). The focus is on **high-quality processing and scalable data integration**, not on data volume.

Technical Implementation and UI

Model Selection

Do **not** use paid APIs. Choose an **open-source LLM** that fits your local resources and briefly justify your choice in the documentation (trade-offs).
If this is not feasible, the use of **dummy LLMs** is also acceptable.

User Interface (UI)

Create a **simplified prototype UI** using **Streamlit** that demonstrates the key steps of the agent's operation and the output of the RAG process.

Runtime Environment

Your solution must be **containerized**. Creating a **Dockerfile** is mandatory.
It is an advantage if solutions with multiple components (e.g. UI, API) are orchestrated using a **docker-compose.yml** file.

Evaluation and Performance Analysis

Functional Evaluation

Compile a **mini evaluation set of 10–20 questions** related to your chosen problem and evaluate your system's performance, either on **a single node** or on the **entire agentic workflow**.

Performance Testing (Load Scenario)

Present a **simplified load test** (50–200 queries). Summarize the results, including:

- Basic latency metrics
- Identification of the system's main bottleneck
- 1–2 concrete optimization recommendations

Deliverables and Evaluation Criteria

Deliverables

- The complete project source code in a **Git repository**
- **Dockerfile** (and optionally **docker-compose.yml**) for reproducibility
- A **README.md** documentation that includes:
    - Description of the problem and objectives
    - Overview of the system architecture and justification of design decisions
    - Summary of functional evaluation and performance test results
    - Installation and execution instructions

Evaluation Criteria

- Code quality and readability
- Reproducibility of the solution
- Relevance and justification of the problem selection
- Quality of the agentic architecture and LangGraph implementation
- Evaluation methodology and conclusions drawn
- Depth of performance analysis and bottleneck identification